Final Project S26

Objectives

With this final project, you have two options:

1. Breadth approach (most common) - based on the requirements listed below, build a project / tool of your choosing. I will not be judging your code (as long as it's not super simple like Hello World or guessing game), instead looking for a **demonstration of flow**.
    1. **"Here is my code."** (Unix/Git)
    2. **"I push it, and it's automatically tested and turned into an image."** (Docker)
    3. **"My infrastructure is managed entirely by code—I never clicked a button in the GCP Console."** (IaC/Terraform)
    4. **"The app is now live at this URL, and I can see the logs here."** (GCP)

Yes, this project is vague. Yes, you can slack off in one category if you blow my socks off with another. The world is your oyster with this project! Hopefully you're not allergic to shellfish.

Git

Required:

- **Repository containing your work**. No, you cannot just push the entire project to it at once, you will need to have multiple commits, and branches.

Optional:

- **Github Actions and testing**, to guarantee your code works as it should. Note: please do not `assert 1+1 = 2`. That is not good testing. For a refresher on actions, check out the repo we covered in class: vanderbran/python-action-example

Docker

Required:

- **Dockerfile** - this will be used to run your code. If you choose one of the other Docker options, this will not be required.
- **Makefile** - so I know what steps to take when running your project.

Optional:

- **Docker development environment** - choose a unix environment (alpine, ubuntu, etc), and build up a development environment from it to run/work with your project in. If this is

chosen, you'll need to install packages and create a user to work within the container. Setting Up a Developer Environment Using Docker

- **Docker Compose (compose.yaml)** - If your application requires multiple containers (frontend, backend, database, etc), you'll need to create a compose file to automatically spin them all up.

Infrastructure as Code

Optional Ansible

- **Create an Ansible script** to deploy your project onto a device (can be localhost, or can be a remote host).
- **Create an Ansible script to automatically provision a GCP virtual machine capable of running your device.** Note: this utilizes the GCloud API, and often requires setup. If you take this path, I will knock one of the other required pieces off.

Google Cloud

Optional:

- **Auto deploy** - setting up GCP to automatically receive an update based on a repo push.
- **Hosting** (non-auto deploy) - manually clone git repo and spin up application.

AI Policy

- As with the rest of this class, the AI policy with this assignment is open, as you may have to work with code/libraries you have not learned yet.
- Keep in mind, with the DevOps side of things, I expect you to be able to explain what you did and why you did it. I will not question things like "why did you choose that database?"

Examples

1. Simple web application in your favorite language - nodejs, python, go, etc...
2. Automatic Docker container provisioning - prompts the user for relevant details (image type, ports, volumes, etc). Creates a development environment with those details.
3. *Cal gave me approval*: If only there was a Python project that would be awesome to do all of this stuff with... like your 2310 project... Put your 2310 project through the DevOps pipeline - building tests, dockerfiles, and setting up auto-deploy on push.

What to Turn In?

1. Repository containing your code, Dockerfiles, etc.
2. If hosted online, a link to the hosting.